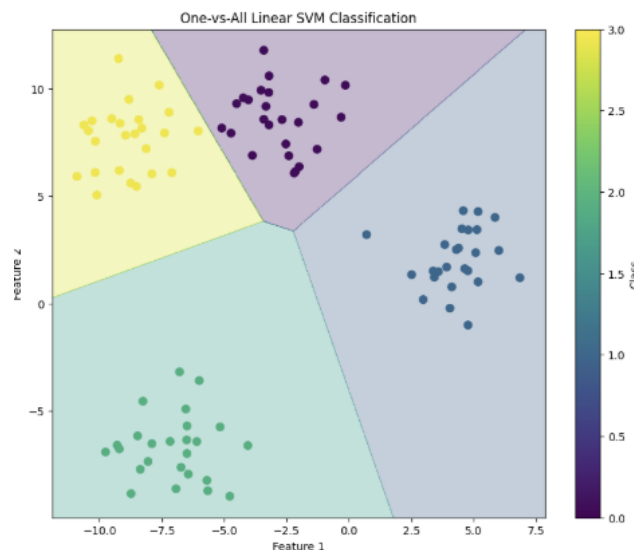


Multiclass Classification with Support Vector Machines

In problems where the number of classes is $K > 2$, there are several ways to construct a multiclass classification model. In the case of a Support Vector Machine, several options have been used, almost all of them based on binary classifiers. The common ones are:

1) One-Against-All (OAA) machines.

In a OAA (also known as one-versus-all, OVA), K independent classifiers are trained. For classifier k , training data of class k is labelled as 1 and the rest of data is labelled as -1. During testing, the model calculates the decision function value for each of the binary SVM classifiers. The class with the highest decision function value is then assigned as the predicted class for that instance. Essentially, each SVM gives a "score" indicating how likely an instance belongs to the positive class for that specific SVM. The one-vs-all method chooses the class whose corresponding SVM gives the highest score. The linear SVM in Scikit-Learn (`sklearn.svm.LinearSVC`) uses this approach

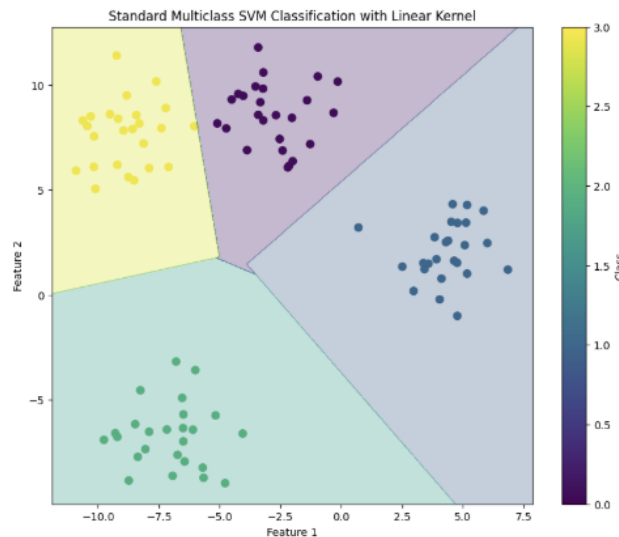


The image above shows an example of OAA classification with data labelled with four different classes. The boundaries have been drawn by computing which one of the four classifiers provides the highest output for each point in the plane. This partitions the space in 4 sections.

2) One Against One (OAO)

The OAO or OVO strategy consists of constructing $k(k-1)/2$ binary classifiers. For example, if there are 4 classes, the OAO strategy constructs 1A2, 1A3, 1A4, 2A3, 2A4 and 3A4 classifiers, this is 6 different classifiers. Classifier 1A2 is trained with classes 1 and 2, where the data of class 1 is labelled as +1 and the data of class 2 is labelled as -1. Classifier 1A3 is trained with classes 1 and 3, labelled as +1 and -1 respectively, and so on. During the test, to predict the class of a new data point, the new point is evaluated by all binary classifiers. Each classifier cast one vote for one of its two classes, and the class that receives the most votes is selected as the prediction for the datapoint. The standard kernel SVM in Scikit-Learn (`sklearn.svm.SVC`) uses this approach.

The figure corresponds to an example of SVM classification that uses the OAO strategy using the multiclass class `sklearn.svm.SVC` with a linear kernel. Its result is actually very similar to the



previous one using the OAA approach.

3) Direct formulation of the multiclass SVM (k-class SVM)

This approach is a modification of the SVM formulation to construct a single classifier able to classify among the different classes. It was introduced in 1998 by J. Weston and C. Watkins in the technical report, *Weston, Jason, and Chris Watkins. Multi-class support vector machines. Technical Report CSD-TR-98-04, Department of Computer Science, Royal Holloway, University of London, May, 1998.*

The table below shows a comparison of advantages and tradeoffs of the three approaches.

The UCI machine learning Repository

The University of California at Irvine Machine Learning Repository [link](#) is an online repository of datasets maintained by the UC Irvine since 1987, to provide a central place for researchers, educators, and students to find and share real-world data for developing and testing machine learning algorithms. Most probably, in the XXI century, this repository is maintained for educational purposes, but no public data about the evolution of citations of the UCI repository is available. It contains over 600 dataset from domains as biology and medicine, social sciences, image Recognition, finance, physics, energy, engineering, and others. Each dataset contains:

1. A data file (CSV or text)
2. A description file explaining:
3. Attributes (features)
4. Task type (classification, regression, clustering)
5. Number of instances
6. References to academic papers using it

Feature	One-versus-All (OvA or AOA)	One-versus-One (OvO or AOO)	k-Class SVM (We- ston & Watkins)
Number of Classifiers	k (linear in k)	$k(k-1)/2$ (quadratic in k)	<i>One single optimization problem solved for k classes simultaneously.</i>
Training Data per Classifier	Large: One class vs. all $k-1$ other classes. Imbalanced training sets are common.	Small: Only samples from the two specific classes. Balanced training sets by nature.	Uses all training data in one formulation.
Training Time	Fastest for training, as only k classifiers are trained.	Slower than OvA due to the quadratic number of classifiers, but each classifier is trained on a smaller subset of the data.	Slowest (in its original form) as it involves solving a single, much larger optimization problem.
Prediction / Testing Time	Fast, as only k decision functions are evaluated.	Slower, as $k(k-1)/2$ decision functions are evaluated, usually combined by majority voting .	Fast, as it involves a single maximum decision function: $\arg \max_i f_i(\mathbf{x})$.

Table 1: Comparison of OvA, OvO, and k-Class SVM methods.

Many of the datasets are available through Scikit-Learn or Pandas, but the safest way is to use the code provided in the UCI repository (if present) or to download directly the .zip file. The following [Google ColabLinks](#) code provides several examples on how to download the data.

The assignment consists of programming a complete experiment of classification with support vector machines comparing different kernels.

The student is required to conduct experiments with several of the UCI standard datasets and provide a report with ALL of the elements described below.

Assignment

6.1: Choose at least 4 databases whose target is multiclass (with more than two classes). Use the filter of the web page to select Task: "classification data" and Python: "Available for Python Import" . The number of available data should be of about 170 datasets. Choose two datasets that you consider easy and two datasets that you consider difficult. Break the data into 80% for training and 20% for test.

Solution 6.1

I labeled Car Evaluation and Wine as "easy" because their class boundaries are relatively clean and the dimensionality is moderate. Dermatology and Glass were considered "difficult" due to higher dimensionality (Dermatology) and overlapping class structure with imbalance (Glass).

Dataset	Samples	Features	Classes	Train/Test
Car Evaluation	1728	6	4	1382 / 346
Dermatology	366	34	6	292 / 74
Glass	214	9	6	171 / 43
Wine	178	13	3	142 / 36

Table 2: Summary of the four multiclass datasets used in this assignment.

6.2: Code as script to conduct a classification experiment to be used by the four datasets. The script must have the following features:

- a) It must be able to perform a v-fold validation for all the parameters of the SVC.
- b) It must be able to compare amongst different kernels. One of the kernels must be linear, another one must be the squared exponential (RBF kernel) and a third kernel must be of your choice amongst the standard kernels coded in the Scikit Learn package.
- c) It must train and validate with the training data (80% of the data) and do a test on the remaining 20% of the data.

Perform experiments with all the datasets and the standard SVM. Write a report that contains, at least, the following:

- 1) Description of the data. The description must include an explanation of why you think the data is easy or difficult to use. If so, what are the challenges that a priori the data shows.
- 2) Description of the experiment, including the cross-validation using v-fold. The description must not be limited to the used functions or classes of the Scikit-Learn package, but it must detail the v-fold mechanism, and the choice of the intervals of the parameters of the SVM, that are the C and the parameters of the kernels.
- 3) Presentation of the results. You can use tables, graphs or confusion matrices. The clarity and significance on the presentation of the results will be graded.
- 4) A discussion including at least the comparison of the results with different kernels, and a comment on the values of the optimal parameters.

Solution 6.2

For each dataset, I trained an SVM classifier using only the 80% training portion. I used 5-fold stratified cross-validation to select hyperparameters. The grid included a linear kernel, an RBF kernel, and a polynomial kernel. For each kernel, I searched over C , γ (when applicable), and the polynomial degree. After CV, the best model was retrained on the full training set and evaluated on the 20% test set. Table 3 summarizes the best kernel, its CV accuracy, and the final test accuracy.

Dataset	Best Kernel	CV Acc.	Test Acc.
Car Evaluation	Poly ($C=10$, $\text{deg}=3$)	0.9986	0.9971
Dermatology	Linear ($C=1$)	0.9760	0.9730
Glass	RBF ($C=10$, $\gamma=0.1$)	0.6956	0.7209
Wine	Linear ($C=0.1$)	0.9510	0.9722

Table 3: Best SVM configuration and performance for each dataset.

Overall, the easy datasets (Car Evaluation and Wine) reached very high accuracy, even with simple kernels. Dermatology performed best with a linear kernel despite its higher dimensionality. Glass

remained the most difficult, and the RBF kernel gave the best (but still moderate) performance due to class overlap.

```
1 import numpy as np
2 import pandas as pd
3
4 from ucimlrepo import fetch_ucirepo
5 from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
6 from sklearn.svm import SVC
7 from sklearn.metrics import accuracy_score, confusion_matrix
8 from sklearn.impute import SimpleImputer
9
10 param_grid = [
11     {
12         "kernel": ["linear"],
13         "C": [0.1, 1, 10, 100],
14     },
15     {
16         "kernel": ["rbf"],
17         "C": [0.1, 1, 10, 100],
18         "gamma": [0.01, 0.1, 1],
19     },
20     {
21         "kernel": ["poly"],
22         "C": [0.1, 1, 10],
23         "degree": [2, 3, 4],
24         "gamma": ["scale", "auto"],
25     },
26 ]
27
28
29 def prepare_features(X_train, X_test):
30     """
31     Preprocess features for SVM:
32     1) One-hot encode categorical features (if inputs are pandas DataFrames).
33     2) Align train/test columns after encoding.
34     3) Cast to float to avoid bool dtype issues.
35     4) Impute missing values (NaN) using 'most_frequent'.
36
37     Returns:
38     X_train_pre, X_test_pre: NumPy arrays ready for SVC.
39     """
40     # Step 1: handle categorical features via one-hot encoding if DataFrame
41     if hasattr(X_train, "select_dtypes"):
42         X_train_enc = pd.get_dummies(X_train)
43         X_test_enc = pd.get_dummies(X_test)
44
45     # Align columns between train and test
46     X_train_enc, X_test_enc = X_train_enc.align(
47         X_test_enc, join="left", axis=1, fill_value=0
48     )
49     else:
50         X_train_enc, X_test_enc = X_train, X_test
51
52     # Step 2: cast to float (bool -> 0/1, ints -> float, etc.)
53     X_train_enc = X_train_enc.astype(float)
```

```

54 X_test_enc = X_test_enc.astype(float)
55
56 # Step 3: impute missing values
57 imputer = SimpleImputer(strategy="most_frequent")
58 X_train_imp = imputer.fit_transform(X_train_enc)
59 X_test_imp = imputer.transform(X_test_enc)
60
61 return X_train_imp, X_test_imp
62
63
64 # -----
65 # 6.1 - load 4 multiclass datasets and make 80/20 splits
66 # -----
67
68 dataset_ids = {
69     "car_evaluation": 19,
70     "dermatology": 33,
71     "glass": 42,
72     "wine": 109,
73 }
74
75 datasets = {}
76 splits = {}
77
78 for name, uci_id in dataset_ids.items():
79     print(f"\n=== {name} (UCI id={uci_id}) ===")
80     ds = fetch_ucirepo(id=uci_id)
81
82     X = ds.data.features
83     y = ds.data.targets.squeeze()
84
85     # Ensure y is 1-D (some targets come as DataFrame with 1 column)
86     if getattr(y, "ndim", 1) > 1:
87         y = y.iloc[:, 0] if hasattr(y, "iloc") else y[:, 0]
88
89     n_samples, n_features = X.shape
90     classes = np.unique(y)
91     n_classes = len(classes)
92
93     print(f"samples: {n_samples}, features: {n_features}, classes: {n_classes}")
94     print("class labels:", classes)
95     assert n_classes > 2, f"{name} is not multiclass"
96
97     # 80/20 stratified split
98     X_train, X_test, y_train, y_test = train_test_split(
99         X,
100         y,
101         test_size=0.2,
102         stratify=y,
103         random_state=42,
104     )
105
106     datasets[name] = ds
107     splits[name] = {

```



```

108     "X_train": X_train,
109     "X_test": X_test,
110     "y_train": y_train,
111     "y_test": y_test,
112 }
113
114 print(f"train shape: {X_train.shape}, test shape: {X_test.shape}")
115
116 # Mark which datasets you consider "easy" vs "difficult"
117 easy_datasets = ["car_evaluation", "wine"]
118 difficult_datasets = ["dermatology", "glass"]
119
120 print("\nEasy datasets:", easy_datasets)
121 print("Difficult datasets:", difficult_datasets)
122
123
124 # -----
125 # 6.2 - classification experiment with SVM + v-fold CV
126 # -----
127
128 results = {}
129
130 for name, split in splits.items():
131     print("\n" + "=" * 70)
132     print(f"SVM experiment for dataset: {name}")
133     print("=" * 70)
134
135     X_train = split["X_train"]
136     X_test = split["X_test"]
137     y_train = split["y_train"]
138     y_test = split["y_test"]
139
140     # Prepare features: encode categoricals + cast to numeric + impute NaNs
141     X_train_pre, X_test_pre = prepare_features(X_train, X_test)
142
143     # GridSearchCV: v-fold cross-validation over all SVC parameters
144     grid = GridSearchCV(
145         estimator=SVC(),
146         param_grid=param_grid,
147         cv=cv,
148         scoring="accuracy",
149         n_jobs=-1,
150         refit=True, # refit best model on the full training set
151         return_train_score=False,
152     )
153
154     grid.fit(X_train_pre, y_train)
155
156     # Best model from CV
157     best_model = grid.best_estimator_
158     best_params = grid.best_params_
159     best_cv_acc = grid.best_score_
160
161     print("Best parameters (v-fold CV):", best_params)

```

```

162 print(f"Best mean CV accuracy: {best_cv_acc:.4f}")
163
164 # Summarize best CV accuracy per kernel (for kernel comparison in report)
165 cv_results = grid.cv_results_
166 kernel_best_cv = {}
167 for mean_score, params in zip(cv_results["mean_test_score"], cv_results["params"]):
168     k = params["kernel"]
169     if k not in kernel_best_cv or mean_score > kernel_best_cv[k]:
170         kernel_best_cv[k] = mean_score
171
172 print("\nBest CV accuracy per kernel:")
173 for k, acc in kernel_best_cv.items():
174     print(f"    {k:>6}: {acc:.4f}")
175
176 # Evaluate on the held-out 20% test set
177 y_pred_test = best_model.predict(X_test_pre)
178 test_acc = accuracy_score(y_test, y_pred_test)
179 cm = confusion_matrix(y_test, y_pred_test)
180
181 print(f"\nTest accuracy (on 20% hold-out set): {test_acc:.4f}")
182 print("Confusion matrix (rows = true, cols = predicted):")
183 print(cm)
184
185 # Store everything for later reporting
186 results[name] = {
187     "best_params": best_params,
188     "best_cv_acc": best_cv_acc,
189     "kernel_best_cv": kernel_best_cv,
190     "test_acc": test_acc,
191     "confusion_matrix": cm,
192 }
193
194 print("\n\n=== Summary of results across all datasets ===")
195 for name, res in results.items():
196     print("\nDataset:", name)
197     print("    Best params:", res["best_params"])
198     print(f"    Best CV accuracy: {res['best_cv_acc']:.4f}")
199     print(f"    Test accuracy:    {res['test_acc']:.4f}")
200     print("    Best CV accuracy per kernel:")
201     for k, acc in res["kernel_best_cv"].items():
202         print(f"        {k:>6}: {acc:.4f}")
203     import numpy as np
204     import pandas as pd
205
206     from ucimlrepo import fetch_ucirepo
207     from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
208     from sklearn.svm import SVC
209     from sklearn.metrics import accuracy_score, confusion_matrix
210     from sklearn.impute import SimpleImputer
211
212     v_folds = 5
213     cv = StratifiedKFold(n_splits=v_folds, shuffle=True, random_state=42)
214
215     param_grid = [

```

```
216 {
217     "kernel": ["linear"],
218     "C": [0.1, 1, 10, 100],
219 },
220 {
221     "kernel": ["rbf"],
222     "C": [0.1, 1, 10, 100],
223     "gamma": [0.01, 0.1, 1],
224 },
225 {
226     "kernel": ["poly"],
227     "C": [0.1, 1, 10],
228     "degree": [2, 3, 4],
229     "gamma": ["scale", "auto"],
230 },
231 ]
232
233
234 def prepare_features(X_train, X_test):
235
236     if hasattr(X_train, "select_dtypes"):
237         X_train_enc = pd.get_dummies(X_train)
238         X_test_enc = pd.get_dummies(X_test)
239
240         X_train_enc, X_test_enc = X_train_enc.align(
241             X_test_enc, join="left", axis=1, fill_value=0
242         )
243     else:
244         X_train_enc, X_test_enc = X_train, X_test
245
246     X_train_enc = X_train_enc.astype(float)
247     X_test_enc = X_test_enc.astype(float)
248
249     imputer = SimpleImputer(strategy="most_frequent")
250     X_train_imp = imputer.fit_transform(X_train_enc)
251     X_test_imp = imputer.transform(X_test_enc)
252
253     return X_train_imp, X_test_imp
254
255 dataset_ids = {
256     "car_evaluation": 19,
257     "dermatology": 33,
258     "glass": 42,
259     "wine": 109,
260 }
261
262 datasets = {}
263 splits = {}
264
265 for name, uci_id in dataset_ids.items():
266     print(f"\n=== {name} (UCI id={uci_id}) ===")
267     ds = fetch_ucirepo(id=uci_id)
268
269     X = ds.data.features
```

```
270 y = ds.data.targets.squeeze()
271
272 if getattr(y, "ndim", 1) > 1:
273     y = y.iloc[:, 0] if hasattr(y, "iloc") else y[:, 0]
274
275 n_samples, n_features = X.shape
276 classes = np.unique(y)
277 n_classes = len(classes)
278
279 print(f"samples: {n_samples}, features: {n_features}, classes: {n_classes}")
280 print("class labels:", classes)
281 assert n_classes > 2, f"{name} is not multiclass"
282
283 X_train, X_test, y_train, y_test = train_test_split(
284     X,
285     y,
286     test_size=0.2,
287     stratify=y,
288     random_state=42,
289 )
290
291 datasets[name] = ds
292 splits[name] = {
293     "X_train": X_train,
294     "X_test": X_test,
295     "y_train": y_train,
296     "y_test": y_test,
297 }
298
299 print(f"train shape: {X_train.shape}, test shape: {X_test.shape}")
300
301 easy_datasets = ["car_evaluation", "wine"]
302 difficult_datasets = ["dermatology", "glass"]
303
304 print("\nEasy datasets:", easy_datasets)
305 print("Difficult datasets:", difficult_datasets)
306
307 results = {}
308
309 for name, split in splits.items():
310     print("\n" + "=" * 70)
311     print(f"SVM experiment for dataset: {name}")
312     print("=" * 70)
313
314     X_train = split["X_train"]
315     X_test = split["X_test"]
316     y_train = split["y_train"]
317     y_test = split["y_test"]
318
319     X_train_pre, X_test_pre = prepare_features(X_train, X_test)
320
321     grid = GridSearchCV(
322         estimator=SVC(),
323         param_grid=param_grid,
```

```

324 cv=cv,
325 scoring="accuracy",
326 n_jobs=-1,
327 refit=True,
328 return_train_score=False,
329 )
330
331 grid.fit(X_train_pre, y_train)
332
333 best_model = grid.best_estimator_
334 best_params = grid.best_params_
335 best_cv_acc = grid.best_score_
336
337 print("Best parameters (v-fold CV):", best_params)
338 print(f"Best mean CV accuracy: {best_cv_acc:.4f}")
339
340 cv_results = grid.cv_results_
341 kernel_best_cv = {}
342 for mean_score, params in zip(cv_results["mean_test_score"], cv_results["params"]):
343     k = params["kernel"]
344     if k not in kernel_best_cv or mean_score > kernel_best_cv[k]:
345         kernel_best_cv[k] = mean_score
346
347 print("\nBest CV accuracy per kernel:")
348 for k, acc in kernel_best_cv.items():
349     print(f"    {k:>6}: {acc:.4f}")
350
351 y_pred_test = best_model.predict(X_test_pre)
352 test_acc = accuracy_score(y_test, y_pred_test)
353 cm = confusion_matrix(y_test, y_pred_test)
354
355 print(f"\nTest accuracy (on 20% hold-out set): {test_acc:.4f}")
356 print("Confusion matrix (rows = true, cols = predicted):")
357 print(cm)
358
359 results[name] = {
360     "best_params": best_params,
361     "best_cv_acc": best_cv_acc,
362     "kernel_best_cv": kernel_best_cv,
363     "test_acc": test_acc,
364     "confusion_matrix": cm,
365 }
366
367 print("\n\n=== Summary of results across all datasets ===")
368 for name, res in results.items():
369     print("\nDataset:", name)
370     print("    Best params:", res["best_params"])
371     print(f"    Best CV accuracy: {res['best_cv_acc']:.4f}")
372     print(f"    Test accuracy:    {res['test_acc']:.4f}")
373     print("    Best CV accuracy per kernel:")
374     for k, acc in res["kernel_best_cv"].items():
375         print(f"        {k:>6}: {acc:.4f}")
376

```